

Mini Project: Student Course Registration & Management System

1. Document Information

Item	Details
Project Title	Student Course Registration & Management System
Project Type	Java Servlet-Based Web Application
Intended Users	Students learning Servlets, JSP, JDBC, Sessions, Cookies, RequestDispatcher, and sendRedirect
Difficulty Level	Moderate
Recommended IDE	Eclipse Enterprise Edition
Server	Apache Tomcat
Database	MySQL
Backend	Java Servlets
View Layer	JSP
Database Connectivity	JDBC

2. Purpose of the Project

The purpose of this mini project is to help students practically understand how a Java web application works using **core Servlet technology**.

This project must demonstrate:

Required Topic	Project Usage
Servlet fundamentals	Request handling using Servlets
Servlet lifecycle	Use of lifecycle methods in selected Servlets
Sessions	Admin login state management
Cookies	Remember admin username
RequestDispatcher	Forwarding data/errors to JSP
sendRedirect	Redirecting after successful actions
JDBC	Connecting Java application with MySQL
CRUD operations	Student, Course, and Registration management

3. Project Overview

The system will allow an **admin** to log in and manage students, courses, and student-course registrations.

The admin will be able to:

Admin Function

- Log in using database credentials
- View dashboard
- Add, view, update, and delete students
- Add, view, update, and delete courses
- Register students for courses
- View student-course registrations
- Update registration status
- Delete registration records
- Log out securely

The project should follow a simple MVC-style structure:

```
Servlet = Controller  
JSP = View  
DAO = Database Logic  
Model = Data Representation
```

4. Scope of the Project

4.1 In Scope

The project must include:

In-Scope Requirement

- Admin login using database credentials
- Session-based authentication
- Remember username functionality using cookies
- Dashboard with summary counts
- Student CRUD operations
- Course CRUD operations
- Student-course registration module

In-Scope Requirement

Registration status update
Logout functionality
Servlet lifecycle demonstration
JDBC database connectivity
DAO-based database operations
RequestDispatcher usage
sendRedirect usage
Basic server-side validation

4.2 Out of Scope

The following are not required:

Out-of-Scope Feature

Spring Boot
Hibernate
REST API
File upload
Email notification
Online payment
Password encryption
Multiple user roles
Advanced frontend frameworks
AJAX-based UI

Important note: Plain-text password storage is allowed only for this beginner-level academic mini project. In real applications, passwords must be hashed and securely stored.

5. User Role

5.1 Admin

The system will have only one user role: **Admin**.

The admin can:

Function

Login
View dashboard
Manage students
Manage courses
Manage registrations
Logout

No student login is required in this mini project.

6. Core Concept Mapping

Concept	Mandatory Usage
Servlet lifecycle	LoginServlet, DashboardServlet, AddStudentServlet, AddCourseServlet
Session	Store logged-in admin username
Cookie	Remember admin username
RequestDispatcher	Forward data and validation errors to JSP
sendRedirect	Redirect after successful login, logout, insert, update, delete
JDBC	DAO classes must perform database operations
CRUD	Student, Course, and Registration modules

7. Navigation Rule

This rule must be followed consistently throughout the project.

Situation	Required Technique
Validation error	RequestDispatcher
Invalid login	RequestDispatcher
Displaying data on JSP	RequestDispatcher
Successful login	sendRedirect
Successful insert	sendRedirect
Successful update	sendRedirect
Successful delete	sendRedirect
Logout	sendRedirect
Unauthorized access	sendRedirect to login page

Explanation

Use **RequestDispatcher** when the same request data must be preserved.

Use **sendRedirect** when the browser should make a fresh request after a successful action.

8. Main Modules

The system must contain the following modules:

Module No.	Module Name
1	Login Module
2	Dashboard Module
3	Student Management Module
4	Course Management Module
5	Student-Course Registration Module
6	Logout Module

9. Functional Requirements

9.1 Login Module

9.1.1 Description

The admin must be able to log in using credentials stored in the database.

9.1.2 Login Fields

Field	Description
Username	Admin username
Password	Admin password
Remember Username	Optional checkbox

9.1.3 Default Admin Credential

A default admin record must be inserted into the database.

Username Password

admin admin123

9.1.4 Successful Login Rules

If the username and password are valid:

Rule

Create a session

Store admin username in session

Store username in cookie if Remember Username is selected

Redirect admin to dashboard

9.1.5 Failed Login Rules

If the username or password is invalid:

Rule

Do not create session

Display error message on login page

Use RequestDispatcher to forward back to login page

9.1.6 Cookie Rule

If Remember Username is selected:

Rule

Store only username in cookie

Do not store password in cookie

Cookie should have expiry time

If Remember Username is not selected:

Rule

Existing remembered username cookie should be deleted

9.2 Dashboard Module

9.2.1 Description

After successful login, the admin must see the dashboard.

9.2.2 Dashboard Content

The dashboard should display:

Dashboard Data

Welcome message

Logged-in admin username

Total number of students

Total number of courses

Total number of registrations

Navigation links to student, course, and registration modules

Logout option

9.2.3 Access Rule

The dashboard must be accessible only after login.

If an unauthenticated user opens the dashboard URL directly:

Expected Behaviour

Redirect user to login page

9.3 Student Management Module

9.3.1 Description

The admin must be able to perform CRUD operations on student records.

9.3.2 Student Fields

Field	Description
Student ID	Auto-generated unique ID
Student Name	Name of the student
Email	Student email

Field	Description
Phone	Student phone number
Age	Student age
City	Student city

9.3.3 Add Student

The admin must be able to add a new student.

Validation Rules

Field	Validation Rule
Student Name	Required
Email	Required
Phone	Required
Age	Must be 18 or above
City	Required

Expected Flow

```
Add Student Form
↓
Submit student details
↓
Validate input
↓
If valid:
  Insert student into database
  Redirect to student list

If invalid:
  Forward back to student form with error message
```

9.3.4 View Students

The admin must be able to view all students.

The student list should display:

Student ID Name Email Phone Age City Actions

Actions:

Action

Edit

Delete

9.3.5 Update Student

The admin must be able to update existing student details.

Expected Flow

```
Student List
↓
Click Edit
↓
Load existing student data
↓
Show update form
↓
Submit updated data
↓
Validate input
↓
If valid:
  Update database
  Redirect to student list
If invalid:
  Forward back to update form with error message
```

9.3.6 Delete Student

The admin must be able to delete a student only if the student is not registered for any course.

Delete Rule

Condition	Expected Behaviour
Student is not registered in any course	Delete student and redirect to student list
Student is already registered in a course	Do not delete and show error message

9.4 Course Management Module

9.4.1 Description

The admin must be able to perform CRUD operations on course records.

9.4.2 Course Fields

Field	Description
Course ID	Auto-generated unique ID
Course Name	Name of the course
Duration	Course duration
Fees	Course fees
Trainer Name	Name of trainer

9.4.3 Add Course

The admin must be able to add a new course.

Validation Rules

Field	Validation Rule
Course Name	Required
Duration	Required
Fees	Must be greater than 0
Trainer Name	Required

Expected Flow

```
Add Course Form
↓
Submit course details
↓
Validate input
↓
If valid:
  Insert course into database
  Redirect to course list

If invalid:
  Forward back to course form with error message
```

9.4.4 View Courses

The admin must be able to view all courses.

The course list should display:

Course ID	Course Name	Duration	Fees	Trainer Name	Actions
-----------	-------------	----------	------	--------------	---------

Actions:

Action

Edit

Delete

9.4.5 Update Course

The admin must be able to update existing course details.

Expected Flow

Course List

↓

Click Edit

↓

Load existing course data

↓

Show update form

↓

Submit updated data

↓

Validate input

↓

If valid:

Update database

Redirect to course list

If invalid:

Forward back to update form with error message

9.4.6 Delete Course

The admin must be able to delete a course only if no student is actively registered in that course.

Delete Rule

Condition	Expected Behaviour
No active registration exists for the course	Delete course and redirect to course list

Condition	Expected Behaviour
Active registration exists for the course	Do not delete and show error message

9.5 Student-Course Registration Module

9.5.1 Description

The admin must be able to register students for courses.

9.5.2 Registration Fields

Field	Description
Registration ID	Auto-generated unique ID
Student	Selected student
Course	Selected course
Registration Date	Date of registration
Status	Active, Completed, or Cancelled

9.5.3 Add Registration

The registration form must contain:

Field
Student dropdown
Course dropdown
Registration date
Status

Dropdown Rules

Dropdown	Source
Student dropdown	Students table
Course dropdown	Courses table

9.5.4 Registration Validation Rules

Field	Validation Rule
Student	Must be selected
Course	Must be selected
Registration Date	Required
Status	Must be Active, Completed, or Cancelled

Duplicate Rule

The same student must not be registered for the same course more than once with **Active** status.

9.5.5 View Registrations

The registration list should display:

Registration ID	Student Name	Course Name	Registration Date	Status	Actions
-----------------	--------------	-------------	-------------------	--------	---------

Actions:

Action

Update Status

Delete

9.5.6 Update Registration Status

The admin must be able to update the registration status.

Allowed statuses:

Status

Active

Completed

Cancelled

After successful status update:

Expected Behaviour

Redirect to registration list

9.5.7 Delete Registration

For this mini project, deleting a registration means removing the registration record from the database.

After successful deletion:

Expected Behaviour

Redirect to registration list

9.6 Logout Module

9.6.1 Description

The admin must be able to log out from the application.

9.6.2 Logout Rules

Rule

Destroy the current session

Redirect to login page

Dashboard and CRUD pages must not be accessible after logout

10. Session Requirements

10.1 Session Creation

A session must be created only after successful login.

Required Session Attribute

Attribute Name	Purpose
loggedInUser	Stores logged-in admin username

Optional Session Attribute

Attribute Name	Purpose
loginTime	Stores login time

10.2 Session Validation

The following pages must be protected:

Protected Page/Action

Dashboard
Student list
Add student
Update student
Delete student
Course list
Add course
Update course
Delete course
Registration form
Registration list
Update registration status
Delete registration
Logout

If session does not exist:

Expected Behaviour

Redirect to login page

10.3 Session Destruction

On logout:

Rule

Invalidate session
Redirect to login page

11. Cookie Requirements

11.1 Cookie Purpose

Cookies must be used only for remembering the admin username.

11.2 Cookie Rules

Rule

Store only username

Do not store password

Cookie should expire after a fixed duration

Cookie should be deleted when Remember Username is unchecked

Login page should display remembered username if cookie exists

12. Servlet Lifecycle Requirements

At least four Servlets must demonstrate lifecycle methods.

12.1 Required Servlets for Lifecycle Demonstration

Servlet

LoginServlet

DashboardServlet

AddStudentServlet

AddCourseServlet

12.2 Lifecycle Methods to Demonstrate

Method	Expected Purpose
init()	Display initialization message in console
doGet()	Handle page loading or fetch request
doPost()	Handle form submission
destroy()	Display destruction message in console

Students must provide console screenshots showing lifecycle messages.

13. Database Requirements

13.1 Database Name

student_course_db

13.2 Required Tables

Table

admin
students
courses
registrations

13.3 Admin Table

Column	Type	Constraint
admin_id	INT	Primary Key, Auto Increment
username	VARCHAR(50)	Not Null, Unique
password	VARCHAR(100)	Not Null

Seed Data

username	password
admin	admin123

13.4 Students Table

Column	Type	Constraint
student_id	INT	Primary Key, Auto Increment
student_name	VARCHAR(100)	Not Null
email	VARCHAR(100)	Not Null
phone	VARCHAR(15)	Not Null
age	INT	Not Null
city	VARCHAR(50)	Not Null

13.5 Courses Table

Column	Type	Constraint
course_id	INT	Primary Key, Auto Increment
course_name	VARCHAR(100)	Not Null
duration	VARCHAR(50)	Not Null
fees	DOUBLE	Not Null
trainer_name	VARCHAR(100)	Not Null

13.6 Registrations Table

Column	Type	Constraint
registration_id	INT	Primary Key, Auto Increment
student_id	INT	Foreign Key references students(student_id)
course_id	INT	Foreign Key references courses(course_id)
registration_date	DATE	Not Null
status	VARCHAR(20)	Not Null

Allowed values for status:

Status

Active

Completed

Cancelled

14. CRUD Requirements

14.1 Student CRUD

Operation	Requirement
Create	Add new student
Read	View all students
Update	Edit existing student
Delete	Delete student if not registered

14.2 Course CRUD

Operation	Requirement
Create	Add new course
Read	View all courses
Update	Edit existing course
Delete	Delete course if not actively assigned

14.3 Registration CRUD

Operation	Requirement
Create	Register student for course
Read	View all registrations
Update	Update registration status
Delete	Delete registration record

15. Suggested URL Mapping

15.1 Public URLs

URL	Purpose
/login	Show login page
/login-action	Process login
/logout	Logout admin

15.2 Protected URLs

URL	Purpose
/dashboard	Show dashboard
/students	View students
/student/add	Show/process add student
/student/edit	Show/process update student
/student/delete	Delete student
/courses	View courses
/course/add	Show/process add course

URL	Purpose
/course/edit	Show/process update course
/course/delete	Delete course
/registrations	View registrations
/registration/add	Show/process registration
/registration/status	Update registration status
/registration/delete	Delete registration

16. Suggested JSP Pages

All JSP files should be placed inside:

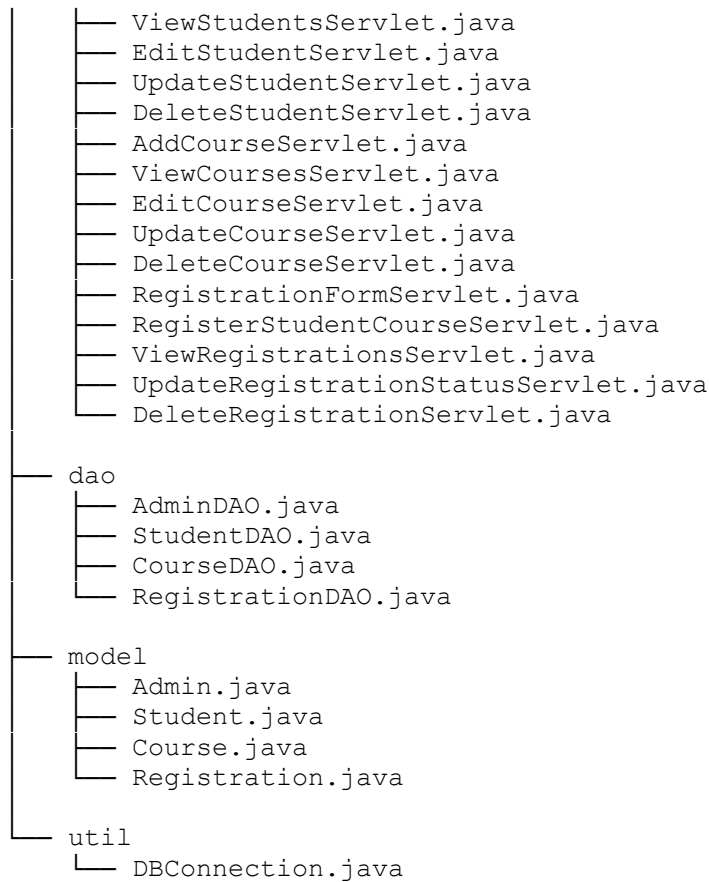
/WEB-INF/views/

JSP Page	Purpose
login.jsp	Login form
dashboard.jsp	Dashboard page
student-form.jsp	Add student form
student-list.jsp	Student list
student-edit.jsp	Update student form
course-form.jsp	Add course form
course-list.jsp	Course list
course-edit.jsp	Update course form
registration-form.jsp	Student-course registration form
registration-list.jsp	Registration list
error.jsp	Common error page

Important: JSP pages inside `WEB-INF/views` must not be directly accessed from the browser. They should be accessed through Servlets only.

17. Suggested Package Structure

```
src/main/java
├── com.studentcourse
│   └── controller
│       ├── LoginPageServlet.java
│       ├── LoginServlet.java
│       ├── LogoutServlet.java
│       ├── DashboardServlet.java
│       └── AddStudentServlet.java
```



18. Database Layer Requirements

The project must use DAO classes for database operations.

Servlets must not directly contain SQL queries.

Layer	Responsibility
Servlet	Request handling and navigation
DAO	SQL and database operations
Model	Data representation
JSP	Display output
DBConnection	Create and return database connection

19. Validation Requirements

19.1 Login Validation

Field	Validation
Username	Required
Password	Required

19.2 Student Validation

Field	Validation
Student Name	Required
Email	Required
Phone	Required
Age	Must be 18 or above
City	Required

19.3 Course Validation

Field	Validation
Course Name	Required
Duration	Required
Fees	Must be greater than 0
Trainer Name	Required

19.4 Registration Validation

Field	Validation
Student	Required
Course	Required
Registration Date	Required
Status	Required
Duplicate Active Registration	Not allowed

20. Error Handling Requirements

The system must handle the following errors:

Error Scenario	Expected Handling
Invalid login	Forward to login page with error
Empty form fields	Forward to same form with error
Invalid age	Forward to student form with error
Invalid fees	Forward to course form with error
Invalid student ID	Forward to error page or list page with message
Invalid course ID	Forward to error page or list page with message
Duplicate active registration	Forward to registration form with error
Database connection failure	Forward to error page
Record not found	Forward to relevant list page with message
Unauthorized access	Redirect to login page
Delete restricted due to linked records	Forward to list page with error

21. Non-Functional Requirements

21.1 Usability

Requirement

Pages should be simple and readable
Forms should have proper labels
Navigation should be clear
Error messages should be understandable
Success messages should be meaningful
Tables should clearly show available actions

21.2 Security

Requirement

Protected pages must not open without login
Password must not be stored in cookies
Session must be destroyed on logout
JSP pages should be inside WEB-INF/views
User input must be validated before database operation
SQL operations must use PreparedStatement
Large or sensitive data should not be stored in session

21.3 Maintainability

Requirement

Servlet should handle request and response flow only
DAO should handle database logic
Model classes should represent data
JSP should display output only
Repeated database connection code should be avoided
Code should use proper naming conventions

22. Expected Application Flows

22.1 Login Flow

```
/login
↓
LoginPageServlet reads remembered username cookie
↓
Forwards to login.jsp
↓
Admin submits login form
↓
LoginServlet validates credentials from database
↓
If valid:
    Create session
    Create/delete cookie based on remember option
    Redirect to dashboard

If invalid:
    Forward to login.jsp with error
```

22.2 Add Student Flow

```
/student/add
↓
Show student form
↓
Submit student details
↓
Validate input
↓
If valid:
    Insert into database
    Redirect to /students
```



```
If invalid:
    Forward to student-form.jsp with error
```

22.3 View Students Flow

```
/students
↓
Fetch student list from database
↓
Forward to student-list.jsp
```

22.4 Update Student Flow

```
/student/edit
↓
Load selected student data
↓
Forward to student-edit.jsp
↓
Submit updated details
↓
Validate input
↓
If valid:
    Update database
    Redirect to /students

If invalid:
    Forward to student-edit.jsp with error
```

22.5 Delete Student Flow

```
/student/delete
↓
Check whether student has registrations
↓
If no registrations:
    Delete student
    Redirect to /students

If registrations exist:
    Do not delete
    Forward to student-list.jsp with error
```

22.6 Add Course Flow

```
/course/add
↓
Show course form
↓
Submit course details
↓
Validate input
↓
If valid:
    Insert into database
    Redirect to /courses

If invalid:
    Forward to course-form.jsp with error
```

22.7 View Courses Flow

```
/courses
↓
Fetch course list from database
↓
Forward to course-list.jsp
```

22.8 Registration Flow

```
/registration/add
↓
Fetch students and courses from database
↓
Forward to registration-form.jsp
↓
Admin selects student, course, date, and status
↓
Validate registration
↓
If valid:
    Insert registration
    Redirect to /registrations

If invalid:
    Forward to registration-form.jsp with error
```

22.9 Logout Flow

```
/logout
↓
Invalidate session
↓
```

Mini Project: Student Course Registration & Management System

1. Document Information

Item	Details
Project Title	Student Course Registration & Management System
Project Type	Java Servlet-Based Web Application
Intended Users	Students learning Servlets, JSP, JDBC, Sessions, Cookies, RequestDispatcher, and sendRedirect
Difficulty Level	Moderate
Recommended IDE	Eclipse Enterprise Edition
Server	Apache Tomcat
Database	MySQL
Backend	Java Servlets
View Layer	JSP
Database Connectivity	JDBC

2. Purpose of the Project

The purpose of this mini project is to help students practically understand how a Java web application works using **core Servlet technology**.

This project must demonstrate:

Required Topic	Project Usage
Servlet fundamentals	Request handling using Servlets
Servlet lifecycle	Use of lifecycle methods in selected Servlets
Sessions	Admin login state management
Cookies	Remember admin username
RequestDispatcher	Forwarding data/errors to JSP
sendRedirect	Redirecting after successful actions
JDBC	Connecting Java application with MySQL
CRUD operations	Student, Course, and Registration management

3. Project Overview

The system will allow an **admin** to log in and manage students, courses, and student-course registrations.

The admin will be able to:

Admin Function

- Log in using database credentials
- View dashboard
- Add, view, update, and delete students
- Add, view, update, and delete courses
- Register students for courses
- View student-course registrations
- Update registration status
- Delete registration records
- Log out securely

The project should follow a simple MVC-style structure:

```
Servlet = Controller  
JSP = View  
DAO = Database Logic  
Model = Data Representation
```

4. Scope of the Project

4.1 In Scope

The project must include:

In-Scope Requirement

- Admin login using database credentials
- Session-based authentication
- Remember username functionality using cookies
- Dashboard with summary counts
- Student CRUD operations
- Course CRUD operations
- Student-course registration module

In-Scope Requirement

Registration status update
Logout functionality
Servlet lifecycle demonstration
JDBC database connectivity
DAO-based database operations
RequestDispatcher usage
sendRedirect usage
Basic server-side validation

4.2 Out of Scope

The following are not required:

Out-of-Scope Feature

Spring Boot
Hibernate
REST API
File upload
Email notification
Online payment
Password encryption
Multiple user roles
Advanced frontend frameworks
AJAX-based UI

Important note: Plain-text password storage is allowed only for this beginner-level academic mini project. In real applications, passwords must be hashed and securely stored.

5. User Role

5.1 Admin

The system will have only one user role: **Admin**.

The admin can:

Function

Login
View dashboard
Manage students
Manage courses
Manage registrations
Logout

No student login is required in this mini project.

6. Core Concept Mapping

Concept	Mandatory Usage
Servlet lifecycle	LoginServlet, DashboardServlet, AddStudentServlet, AddCourseServlet
Session	Store logged-in admin username
Cookie	Remember admin username
RequestDispatcher	Forward data and validation errors to JSP
sendRedirect	Redirect after successful login, logout, insert, update, delete
JDBC	DAO classes must perform database operations
CRUD	Student, Course, and Registration modules

7. Navigation Rule

This rule must be followed consistently throughout the project.

Situation	Required Technique
Validation error	RequestDispatcher
Invalid login	RequestDispatcher
Displaying data on JSP	RequestDispatcher
Successful login	sendRedirect
Successful insert	sendRedirect
Successful update	sendRedirect
Successful delete	sendRedirect
Logout	sendRedirect
Unauthorized access	sendRedirect to login page

Explanation

Use **RequestDispatcher** when the same request data must be preserved.

Use **sendRedirect** when the browser should make a fresh request after a successful action.

8. Main Modules

The system must contain the following modules:

Module No.	Module Name
1	Login Module
2	Dashboard Module
3	Student Management Module
4	Course Management Module
5	Student-Course Registration Module
6	Logout Module

9. Functional Requirements

9.1 Login Module

9.1.1 Description

The admin must be able to log in using credentials stored in the database.

9.1.2 Login Fields

Field	Description
Username	Admin username
Password	Admin password
Remember Username	Optional checkbox

9.1.3 Default Admin Credential

A default admin record must be inserted into the database.

Username Password

admin admin123

9.1.4 Successful Login Rules

If the username and password are valid:

Rule

Create a session

Store admin username in session

Store username in cookie if Remember Username is selected

Redirect admin to dashboard

9.1.5 Failed Login Rules

If the username or password is invalid:

Rule

Do not create session

Display error message on login page

Use RequestDispatcher to forward back to login page

9.1.6 Cookie Rule

If Remember Username is selected:

Rule

Store only username in cookie

Do not store password in cookie

Cookie should have expiry time

If Remember Username is not selected:

Rule

Existing remembered username cookie should be deleted

9.2 Dashboard Module

9.2.1 Description

After successful login, the admin must see the dashboard.

9.2.2 Dashboard Content

The dashboard should display:

Dashboard Data

Welcome message

Logged-in admin username

Total number of students

Total number of courses

Total number of registrations

Navigation links to student, course, and registration modules

Logout option

9.2.3 Access Rule

The dashboard must be accessible only after login.

If an unauthenticated user opens the dashboard URL directly:

Expected Behaviour

Redirect user to login page

9.3 Student Management Module

9.3.1 Description

The admin must be able to perform CRUD operations on student records.

9.3.2 Student Fields

Field	Description
Student ID	Auto-generated unique ID
Student Name	Name of the student
Email	Student email

Field	Description
Phone	Student phone number
Age	Student age
City	Student city

9.3.3 Add Student

The admin must be able to add a new student.

Validation Rules

Field	Validation Rule
Student Name	Required
Email	Required
Phone	Required
Age	Must be 18 or above
City	Required

Expected Flow

```
Add Student Form
↓
Submit student details
↓
Validate input
↓
If valid:
  Insert student into database
  Redirect to student list

If invalid:
  Forward back to student form with error message
```

9.3.4 View Students

The admin must be able to view all students.

The student list should display:

Student ID Name Email Phone Age City Actions

Actions:

Action

Edit

Delete

9.3.5 Update Student

The admin must be able to update existing student details.

Expected Flow

```
Student List
↓
Click Edit
↓
Load existing student data
↓
Show update form
↓
Submit updated data
↓
Validate input
↓
If valid:
    Update database
    Redirect to student list
If invalid:
    Forward back to update form with error message
```

9.3.6 Delete Student

The admin must be able to delete a student only if the student is not registered for any course.

Delete Rule

Condition	Expected Behaviour
Student is not registered in any course	Delete student and redirect to student list
Student is already registered in a course	Do not delete and show error message

9.4 Course Management Module

9.4.1 Description

The admin must be able to perform CRUD operations on course records.

9.4.2 Course Fields

Field	Description
Course ID	Auto-generated unique ID
Course Name	Name of the course
Duration	Course duration
Fees	Course fees
Trainer Name	Name of trainer

9.4.3 Add Course

The admin must be able to add a new course.

Validation Rules

Field	Validation Rule
Course Name	Required
Duration	Required
Fees	Must be greater than 0
Trainer Name	Required

Expected Flow

```
Add Course Form
↓
Submit course details
↓
Validate input
↓
If valid:
  Insert course into database
  Redirect to course list

If invalid:
  Forward back to course form with error message
```

9.4.4 View Courses

The admin must be able to view all courses.

The course list should display:

Course ID	Course Name	Duration	Fees	Trainer Name	Actions
-----------	-------------	----------	------	--------------	---------

Actions:

Action

Edit

Delete

9.4.5 Update Course

The admin must be able to update existing course details.

Expected Flow

Course List

↓

Click Edit

↓

Load existing course data

↓

Show update form

↓

Submit updated data

↓

Validate input

↓

If valid:

Update database

Redirect to course list

If invalid:

Forward back to update form with error message

9.4.6 Delete Course

The admin must be able to delete a course only if no student is actively registered in that course.

Delete Rule

Condition	Expected Behaviour
No active registration exists for the course	Delete course and redirect to course list

Condition	Expected Behaviour
Active registration exists for the course	Do not delete and show error message

9.5 Student-Course Registration Module

9.5.1 Description

The admin must be able to register students for courses.

9.5.2 Registration Fields

Field	Description
Registration ID	Auto-generated unique ID
Student	Selected student
Course	Selected course
Registration Date	Date of registration
Status	Active, Completed, or Cancelled

9.5.3 Add Registration

The registration form must contain:

Field
Student dropdown
Course dropdown
Registration date
Status

Dropdown Rules

Dropdown	Source
Student dropdown	Students table
Course dropdown	Courses table

9.5.4 Registration Validation Rules

Field	Validation Rule
Student	Must be selected
Course	Must be selected
Registration Date	Required
Status	Must be Active, Completed, or Cancelled

Duplicate Rule

The same student must not be registered for the same course more than once with **Active** status.

9.5.5 View Registrations

The registration list should display:

Registration ID	Student Name	Course Name	Registration Date	Status	Actions
-----------------	--------------	-------------	-------------------	--------	---------

Actions:

Action
Update Status
Delete

9.5.6 Update Registration Status

The admin must be able to update the registration status.

Allowed statuses:

Status
Active
Completed
Cancelled

After successful status update:

Expected Behaviour
Redirect to registration list

9.5.7 Delete Registration

For this mini project, deleting a registration means removing the registration record from the database.

After successful deletion:

Expected Behaviour

Redirect to registration list

9.6 Logout Module

9.6.1 Description

The admin must be able to log out from the application.

9.6.2 Logout Rules

Rule

Destroy the current session

Redirect to login page

Dashboard and CRUD pages must not be accessible after logout

10. Session Requirements

10.1 Session Creation

A session must be created only after successful login.

Required Session Attribute

Attribute Name	Purpose
loggedInUser	Stores logged-in admin username

Optional Session Attribute

Attribute Name	Purpose
loginTime	Stores login time

10.2 Session Validation

The following pages must be protected:

Protected Page/Action

Dashboard
Student list
Add student
Update student
Delete student
Course list
Add course
Update course
Delete course
Registration form
Registration list
Update registration status
Delete registration
Logout

If session does not exist:

Expected Behaviour

Redirect to login page

10.3 Session Destruction

On logout:

Rule

Invalidate session
Redirect to login page

11. Cookie Requirements

11.1 Cookie Purpose

Cookies must be used only for remembering the admin username.

11.2 Cookie Rules

Rule

Store only username

Do not store password

Cookie should expire after a fixed duration

Cookie should be deleted when Remember Username is unchecked

Login page should display remembered username if cookie exists

12. Servlet Lifecycle Requirements

At least four Servlets must demonstrate lifecycle methods.

12.1 Required Servlets for Lifecycle Demonstration

Servlet

LoginServlet

DashboardServlet

AddStudentServlet

AddCourseServlet

12.2 Lifecycle Methods to Demonstrate

Method	Expected Purpose
init()	Display initialization message in console
doGet()	Handle page loading or fetch request
doPost()	Handle form submission
destroy()	Display destruction message in console

Students must provide console screenshots showing lifecycle messages.

13. Database Requirements

13.1 Database Name

student_course_db

13.2 Required Tables

Table

admin
students
courses
registrations

13.3 Admin Table

Column	Type	Constraint
admin_id	INT	Primary Key, Auto Increment
username	VARCHAR(50)	Not Null, Unique
password	VARCHAR(100)	Not Null

Seed Data

username	password
admin	admin123

13.4 Students Table

Column	Type	Constraint
student_id	INT	Primary Key, Auto Increment
student_name	VARCHAR(100)	Not Null
email	VARCHAR(100)	Not Null
phone	VARCHAR(15)	Not Null
age	INT	Not Null
city	VARCHAR(50)	Not Null

13.5 Courses Table

Column	Type	Constraint
course_id	INT	Primary Key, Auto Increment
course_name	VARCHAR(100)	Not Null
duration	VARCHAR(50)	Not Null
fees	DOUBLE	Not Null
trainer_name	VARCHAR(100)	Not Null

13.6 Registrations Table

Column	Type	Constraint
registration_id	INT	Primary Key, Auto Increment
student_id	INT	Foreign Key references students(student_id)
course_id	INT	Foreign Key references courses(course_id)
registration_date	DATE	Not Null
status	VARCHAR(20)	Not Null

Allowed values for status:

Status

Active

Completed

Cancelled

14. CRUD Requirements

14.1 Student CRUD

Operation	Requirement
Create	Add new student
Read	View all students
Update	Edit existing student
Delete	Delete student if not registered

14.2 Course CRUD

Operation	Requirement
Create	Add new course
Read	View all courses
Update	Edit existing course
Delete	Delete course if not actively assigned

14.3 Registration CRUD

Operation	Requirement
Create	Register student for course
Read	View all registrations
Update	Update registration status
Delete	Delete registration record

15. Suggested URL Mapping

15.1 Public URLs

URL	Purpose
/login	Show login page
/login-action	Process login
/logout	Logout admin

15.2 Protected URLs

URL	Purpose
/dashboard	Show dashboard
/students	View students
/student/add	Show/process add student
/student/edit	Show/process update student
/student/delete	Delete student
/courses	View courses
/course/add	Show/process add course

URL	Purpose
/course/edit	Show/process update course
/course/delete	Delete course
/registrations	View registrations
/registration/add	Show/process registration
/registration/status	Update registration status
/registration/delete	Delete registration

16. Suggested JSP Pages

All JSP files should be placed inside:

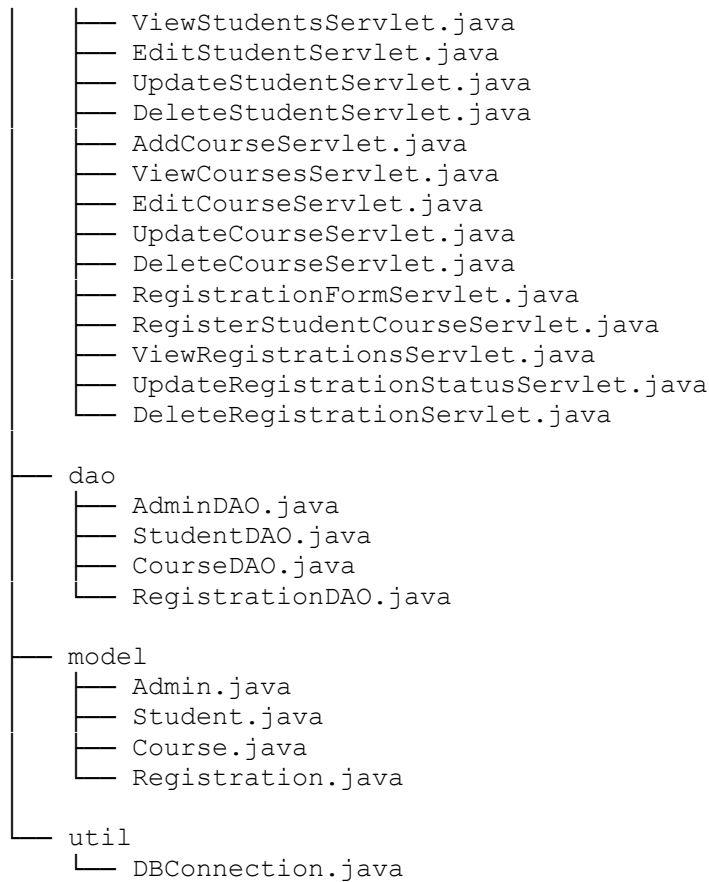
/WEB-INF/views/

JSP Page	Purpose
login.jsp	Login form
dashboard.jsp	Dashboard page
student-form.jsp	Add student form
student-list.jsp	Student list
student-edit.jsp	Update student form
course-form.jsp	Add course form
course-list.jsp	Course list
course-edit.jsp	Update course form
registration-form.jsp	Student-course registration form
registration-list.jsp	Registration list
error.jsp	Common error page

Important: JSP pages inside `WEB-INF/views` must not be directly accessed from the browser. They should be accessed through Servlets only.

17. Suggested Package Structure

```
src/main/java
├── com.studentcourse
│   └── controller
│       ├── LoginPageServlet.java
│       ├── LoginServlet.java
│       ├── LogoutServlet.java
│       ├── DashboardServlet.java
│       └── AddStudentServlet.java
```



18. Database Layer Requirements

The project must use DAO classes for database operations.

Servlets must not directly contain SQL queries.

Layer	Responsibility
Servlet	Request handling and navigation
DAO	SQL and database operations
Model	Data representation
JSP	Display output
DBConnection	Create and return database connection

19. Validation Requirements

19.1 Login Validation

Field	Validation
Username	Required
Password	Required

19.2 Student Validation

Field	Validation
Student Name	Required
Email	Required
Phone	Required
Age	Must be 18 or above
City	Required

19.3 Course Validation

Field	Validation
Course Name	Required
Duration	Required
Fees	Must be greater than 0
Trainer Name	Required

19.4 Registration Validation

Field	Validation
Student	Required
Course	Required
Registration Date	Required
Status	Required
Duplicate Active Registration	Not allowed

20. Error Handling Requirements

The system must handle the following errors:

Error Scenario	Expected Handling
Invalid login	Forward to login page with error
Empty form fields	Forward to same form with error
Invalid age	Forward to student form with error
Invalid fees	Forward to course form with error
Invalid student ID	Forward to error page or list page with message
Invalid course ID	Forward to error page or list page with message
Duplicate active registration	Forward to registration form with error
Database connection failure	Forward to error page
Record not found	Forward to relevant list page with message
Unauthorized access	Redirect to login page
Delete restricted due to linked records	Forward to list page with error

21. Non-Functional Requirements

21.1 Usability

Requirement

Pages should be simple and readable
Forms should have proper labels
Navigation should be clear
Error messages should be understandable
Success messages should be meaningful
Tables should clearly show available actions

21.2 Security

Requirement

Protected pages must not open without login
Password must not be stored in cookies
Session must be destroyed on logout
JSP pages should be inside WEB-INF/views
User input must be validated before database operation
SQL operations must use PreparedStatement
Large or sensitive data should not be stored in session

21.3 Maintainability

Requirement

Servlet should handle request and response flow only
DAO should handle database logic
Model classes should represent data
JSP should display output only
Repeated database connection code should be avoided
Code should use proper naming conventions

22. Expected Application Flows

22.1 Login Flow

```
/login
↓
LoginPageServlet reads remembered username cookie
↓
Forwards to login.jsp
↓
Admin submits login form
↓
LoginServlet validates credentials from database
↓
If valid:
    Create session
    Create/delete cookie based on remember option
    Redirect to dashboard

If invalid:
    Forward to login.jsp with error
```

22.2 Add Student Flow

```
/student/add
↓
Show student form
↓
Submit student details
↓
Validate input
↓
If valid:
    Insert into database
    Redirect to /students
```

```
If invalid:
    Forward to student-form.jsp with error
```

22.3 View Students Flow

```
/students
↓
Fetch student list from database
↓
Forward to student-list.jsp
```

22.4 Update Student Flow

```
/student/edit
↓
Load selected student data
↓
Forward to student-edit.jsp
↓
Submit updated details
↓
Validate input
↓
If valid:
    Update database
    Redirect to /students

If invalid:
    Forward to student-edit.jsp with error
```

22.5 Delete Student Flow

```
/student/delete
↓
Check whether student has registrations
↓
If no registrations:
    Delete student
    Redirect to /students

If registrations exist:
    Do not delete
    Forward to student-list.jsp with error
```

22.6 Add Course Flow

```
/course/add
↓
Show course form
↓
Submit course details
↓
Validate input
↓
If valid:
    Insert into database
    Redirect to /courses

If invalid:
    Forward to course-form.jsp with error
```

22.7 View Courses Flow

```
/courses
↓
Fetch course list from database
↓
Forward to course-list.jsp
```

22.8 Registration Flow

```
/registration/add
↓
Fetch students and courses from database
↓
Forward to registration-form.jsp
↓
Admin selects student, course, date, and status
↓
Validate registration
↓
If valid:
    Insert registration
    Redirect to /registrations

If invalid:
    Forward to registration-form.jsp with error
```

22.9 Logout Flow

```
/logout
↓
Invalidate session
↓
Redirect to /login
```

